

Research Report

Live Testing, AI Enhancement & Background Traffic Simulation

1. Objective and Scope

The primary goal of this phase was to transition the theoretical ID3 Decision Tree model from a static environment into a live, real-time Deep Packet Inspection (DPI) testing environment. This week focused on three core milestones: upgrading the AI's feature extraction logic to drastically reduce False Positives, integrating the Python engine with Suricata's real-time packet logs, and aggressively stress-testing the system using an automated Bash script that simulates both benign business traffic and chaotic malware generation.

2. AI Model and Feature Engineering Advancements (Python Script)

The initial AI model relied on a 2-Dimensional vector consisting only of *Subdomain Length* and *Shannon Entropy*. While effective in isolation, live network environments contain naturally long, human-readable domains that trigger False Positives. To fortify the mathematical boundary, the Python script was upgraded to utilize a 4-Dimensional Feature Vector:

- **Digit-to-Letter Ratio:** Malicious payloads generated by tools like dnscat2 heavily rely on Base32 or Base64 encoding, resulting in an unnatural density of numbers. By implementing Regular Expressions (`re.findall(r'\d')`), the script now precisely calculates the percentage of digits within the subdomain, creating a stark contrast between an encoded payload and a long Vietnamese domain (e.g., *chuong-trinh-khuyen-mai*).
- **Root Domain Tracking:** Threat actors frequently utilize disposable, randomly generated, or unusually short Top-Level Domains (TLDs/DGAs). By integrating the `tlldextract` Python library, the system now explicitly isolates the Root Domain and measures its length, providing the Decision Tree with critical contextual data regarding domain reputation.
- **Model Restructuring:** The mock training dataset (`X_train`) within the `scikit-learn` module was rewritten to accommodate these four features, allowing the ID3 algorithm to draw a much sharper, multi-variable decision boundary between label 0 (Safe) and label 1 (Malware).

3. Real-Time DPI Integration & Suricata 8.0 Compatibility

To achieve real-time interception, the Python engine was engineered to act as a continuous tailing listener (`f.seek(0, os.SEEK_END)`) on Suricata's `eve.json` output.

During live testing, a critical bug was discovered regarding Suricata version 8.0's updated JSON schema. Unlike older versions, Suricata 8.0 nests the queried domain name inside a queries array, causing the initial script to extract empty strings, calculate an Entropy of 0.0, and flood the console with empty [SAFE] logs. The Python parser was successfully patched to intelligently navigate the nested JSON structure (`dns_info["queries"][0].get("rrname", "")`), ensuring 100% accurate extraction of all A, AAAA, TXT, and CNAME records without dropping packets.

4. Active Mitigation and the "Whitelist" DevSecOps Logic

Upon detecting a malware signature, the Python script executes dynamic OS-level commands to isolate the infected endpoint. To ensure complete isolation, the mitigation function was upgraded to drop packets on both the FORWARD chain (preventing external C&C communication) and the INPUT chain (preventing direct queries to the Gateway). During bidirectional traffic analysis, a dangerous logical flaw was identified: the AI analyzed the outbound *response* packets from the Gateway and subsequently attempted to block the Gateway's own IP address, effectively triggering a "Self-Lockout". This was immediately resolved by hardcoding a Gateway Whitelist (`GATEWAY_IP = "192.168.50.1"`) into the mitigation function, demonstrating a crucial understanding of practical DevSecOps and firewall architecture.

5. Automated Traffic Simulation (The Bash Script Generation)

To empirically prove the model's accuracy, a custom Bash script (`traffic_simulator.sh`) was deployed on the Victim VM to simulate a heavy corporate network load. The script was programmed to generate 100 concurrent DNS requests routed directly to the Gateway:

- **Phase 1: 50 Clean Domains (Baseline Setup):** The script iterates through an array of popular, legitimate domains (e.g., www.google.com, www.vnexpress.net, github.com). This proves the model's ability to process normal traffic without triggering False Positives.
- **Phase 2: 50 Suspicious Domains (DGA Simulation):** To simulate malware, the script utilizes the Linux `/dev/urandom` utility, piped through `tr -dc 'a-z0-9'`, to autonomously generate 50 highly chaotic, 20-to-30 character Domain Generation Algorithm (DGA) strings. These strings are appended to the `.malware.local` zone to trigger the DNS Trap.
- *Resource Optimization:* A deliberate sleep interval was programmed into the bash loops to prevent CPU/RAM choking on the virtualized Gateway, ensuring Suricata and the AI had sufficient hardware resources to parse every packet.

6. Evaluation, False Positives, and Future Work

The simulation results were highly successful. The Python engine effortlessly filtered the 50 clean domains while instantly triggering the [MALWARE ALERT] and executing iptables drops against all 50 generated DGA strings.

Interestingly, the empirical test revealed a highly valuable False Positive edge-case. The AI erroneously flagged chat.openai.com.cdn.cloudflare.net as malware. Because Content Delivery Networks (CDNs) utilize exceptionally long, high-entropy CNAME records for load balancing, their mathematical signature closely mirrors that of DNS Tunneling exfiltration. This scenario perfectly dictates the direction for Future Work: The theoretical "Mock Dataset" currently hardcoded in the Python script must be replaced with a massive, real-world CSV dataset. By ingesting thousands of legitimate CDN signatures into the training phase, the Machine Learning model will adjust its weights, ultimately refining its decision boundary to tolerate CDN infrastructure while maintaining strict zero-trust against actual tunneling payloads.